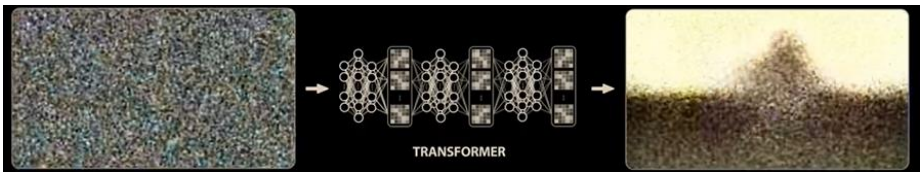
How do AI images and videos actually work? (Welch Labs)

Over the last few years, AI systems have become astonishingly good at turning text props into videos. At the core of how these models operate is a deep connection to physics. This generation of image and video models works using a process known as diffusion, which is remarkably equivalent to the Brownian motion we see as particles diffuse, but with time run backwards, and in high-dimensional space.

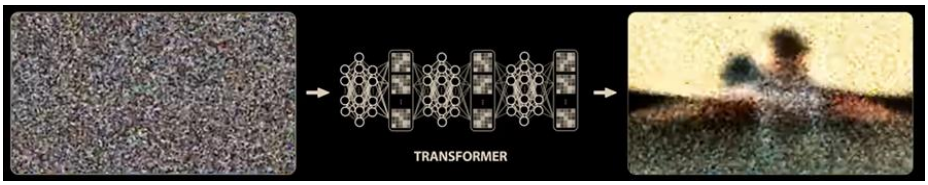
Introduction

While the best models are closed source, there are some compelling open source models (WAN 2.1.) For example, we can add to our prompt and have our astronaut hold a flag, hold a laptop, or hold a meeting.

If we dig into our WAN model's source code, we'll find that the video generation process begins with this call to a random number generator. Creating a video where the pixel intensity values are chosen randomly.
From here, this pure noise video is passed into a transformer. This is the same type of AI model used by large language models, like ChatGPT. But instead of outputting text, this transformer outputs another video that now looks like this.



Still mostly noise, but with some hints of structure. This new video is added to our pure noise video, and then passed back into the model again, producing a third video that looks like this.



This process is repeated again and again. Here's what the video looks like after 50 iterations.

Step by step, our transformer shapes pure noise into incredibly realistic video. But what exactly is the connection to Brownian motion here? And how is our model able to use text input so expressively to shape noise into what our prompt describes?

We'll impact diffusion models in 3 parts:

1) We'll look at a 2021 OpenAI paper and model called CLIP. As we'll see, CLIP is really two models, a **language** model and a **vision** model, that are trained using a clever learning objective that allows them to learn this really powerful shared space between words and pictures. Experimenting with this space will help us get a feel for the high dimensional spaces that diffusion models operate in. But learning a shared representation is not enough to generate images.
2) We'll look at the diffusion process itself. At a high level, **diffusion models are trained to remove noise from images or videos**. However, if you dig into the landmark papers in the field, you'll find that this naive understanding of diffusion really doesn't hold up in practice. In this section we'll dig into the

connection between diffusion models and diffusion processes in physics. This connection will help us understand how these models really work in practice and give us some powerful theory for dramatically speeding up image and video generation.

3) We'll bring these worlds together and see how approaches like CLIP are combined with diffusion models to condition and guide the generation process towards the videos we ask for in our prompts.

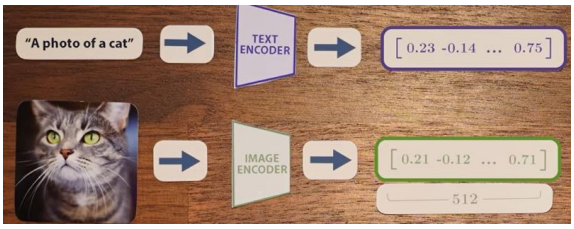
CLIP (Contrastive Language-Image Pre-training)

In February 2021, a team at OpenAI released a new model architecture called CLIP, trained on a dataset of 400 million image and caption pairs scraped from the internet.

CLIP is composed of two models:

- one that processes text
- one that processes images

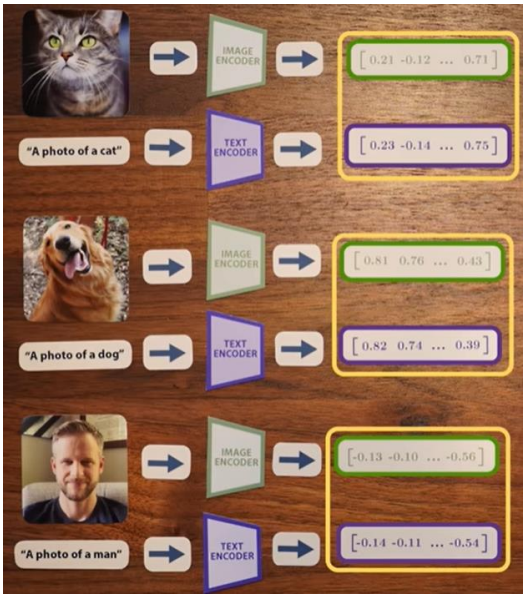
Example:



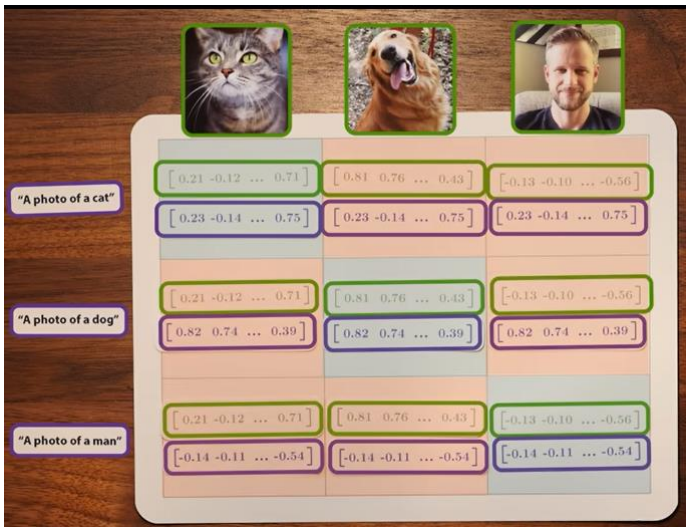
The output of each of these models is a vector of length 512, and the central idea is that the vectors for a given image and its captions **should be similar**.

To achieve this, the OpenAI team developed a clever training approach. Given a batch of image-caption pairs, for example our batch could contain a picture of a cat, a dog, and me, with the captions a photo of a cat, a photo of a dog, and a photo of a man, we then pass our three images into our image model, and our three captions into our text model.

We now have three image vectors and three text vectors, and we would like the vectors for the matching image-caption pairs to be similar.



The clever idea from here is to make use of the **similarity** not just between the corresponding images and captions, but **between all image-caption pairs** in the batch when training our models. If we arrange our image vectors as the columns of a matrix, and our text vectors as the rows, the pairs of vectors along the *diagonal* of our matrix correspond to **matching images** and captions. And all the pairs off-diagonal are **non-matching images** and captions.

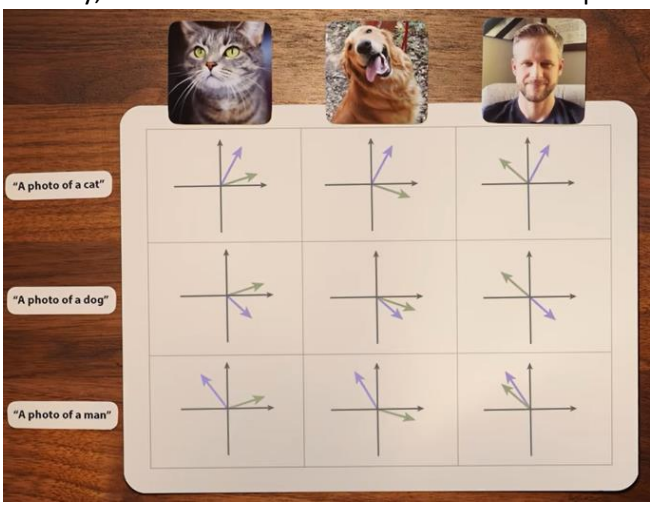


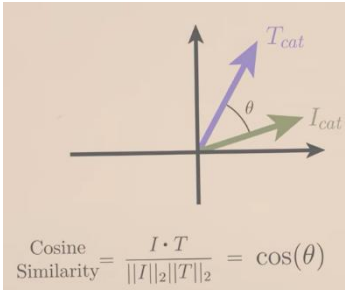
The CLIP training objective seeks to:

- **maximize** the similarity between *corresponding image-caption pairs*,
- while simultaneously **minimizing** the similarity between *non-corresponding image-caption pairs*.

The C in CLIP stands for contrastive because the model learns to **contrast** matching and non-matching image-caption pairs.

The CLIP algorithm measures similarity between vectors using a metric called **cosine similarity**. Geometrically, we can think of each of these vectors as pointing in some direction in high-dimensional space.

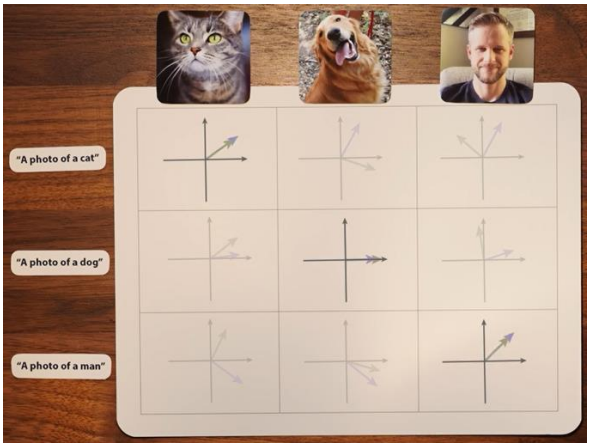




Cosine similarity measures the cosine of the angle between our vectors in this space. So if our text and image vector point in the same direction, the angle between our vectors will be zero, resulting in a maximum value for our cosine similarity score of 1.

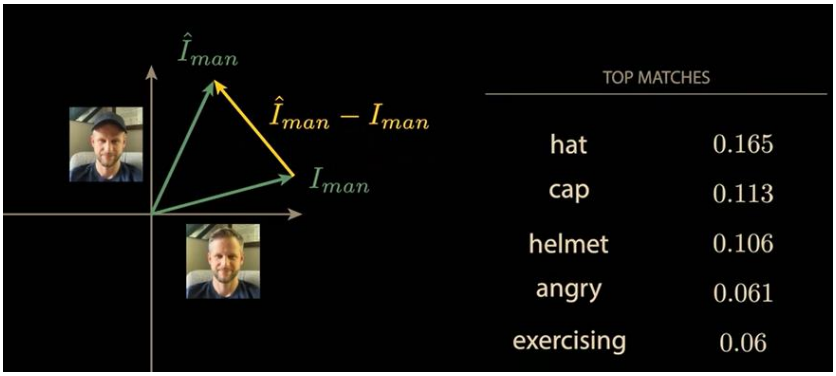
So, the image and text models that make up CLIP are trained to:

- **maximize the alignment of related images** and captions in this shared high-dimensional space
- while **minimizing the alignment between unrelated** images and captions.



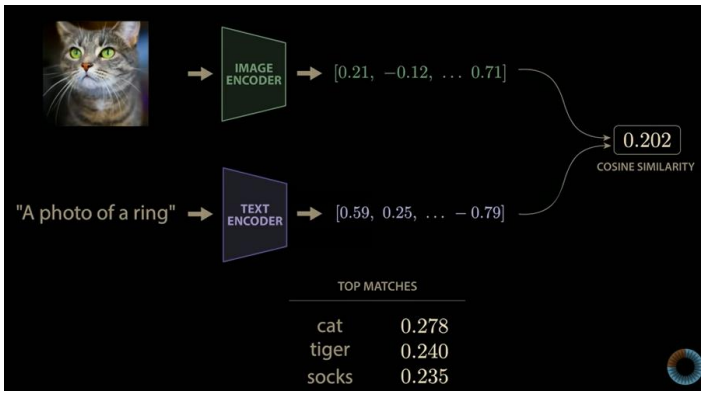
The learned geometry of this shared vector space, known as a **latent** or **embedding space**, has some really interesting properties.

If I take two pictures of myself, one not wearing a hat and one wearing a hat, and pass both of these into our CLIP image model, we get two vectors in our embedding space. Now if I take the vector corresponding to me wearing a hat and *subtract* the vector of me not wearing a hat, we get a new vector in our embedding space. Now what text might this new vector correspond to? Mathematically we took the difference of me wearing a hat and me not wearing a hat. We can search for corresponding text by passing a bunch of different words into our text encoder, and for each computing the cosine similarity between our newly computed difference vector and the text vector. Testing a set of a few hundred common words, the top ranked match with a similarity of 0.165 is the word hat, followed by cap and helmet. This is a remarkable result.



The learned geometry of CLIP's embedding space allows us to operate mathematically on the pure ideas or concepts in our images and text, translating the differences in the content of our images, like if there's a hat or not, into a literal distance between vectors in our embedding space.

The OpenAI team showed that CLIP could produce very impressive image classification results by simply passing an image into our image encoder, and then comparing the resulting vector to a set of possible captions, one for each label that could be assigned to the image, and classifying the image with whatever label resulted in the highest cosine similarity.

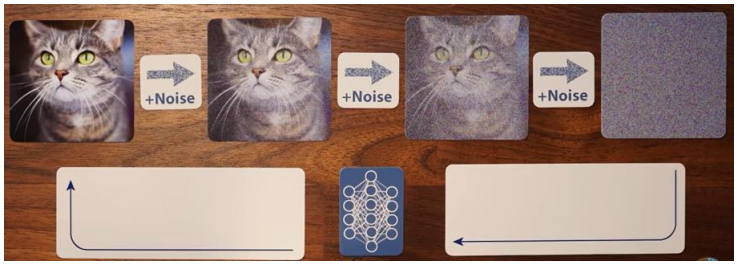


So, techniques like CLIP give us a powerful shared representation of image and text, a kind of vector space of pure ideas. However, our CLIP models only go one direction. We can only map image and text to our shared embedding space. We have no way of generating images and text from our embedding vectors.

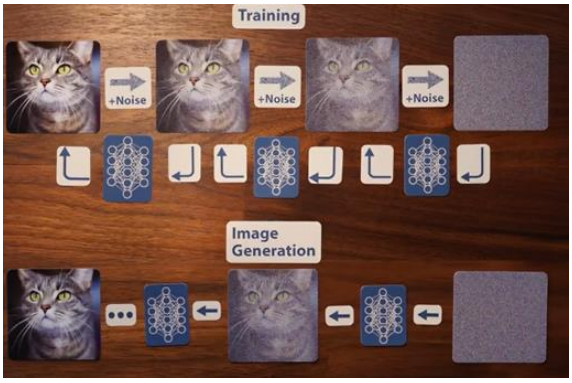
Diffusion model

2020 turned out not only to be a transformative year for language modelling. A few weeks after the GPT-3 paper came out, a team at Berkeley published a paper called Denoising Diffusion Probabilistic Models, now known as DDPM. The paper showed for the first time that it was possible **to generate very high quality images using a diffusion process**, where pure noise is transformed step by step into realistic images.

The core idea behind diffusion models is pretty straightforward. We take a set of training images and add noise to each image step by step until the image is completely destroyed. From here we train a neural network to reverse this process.



When I first learned about diffusion models, I assumed that the models would be trained to remove noise a single step at a time. Our model would be trained to predict the image in step 1 given the noisier image in step 2, trained to predict the image in step 2 given the noisier image in step 3, and so on. When it came time to generate an image, we would pass pure noise into our model, take its output and pass it back into its input again and again, and after enough steps we would have a nice image.



Now, it turns out that this naive approach to building a diffusion model really does not work well. Virtually no modern models work like this.

Algorithm 1 Training

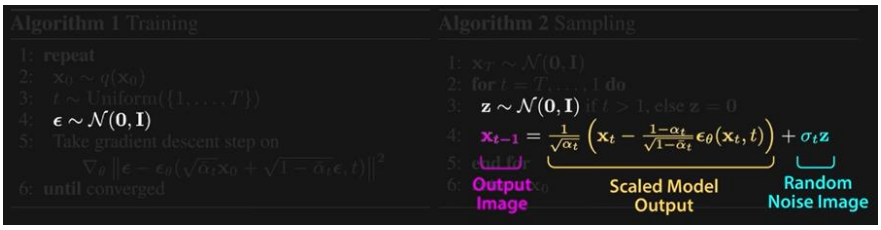
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

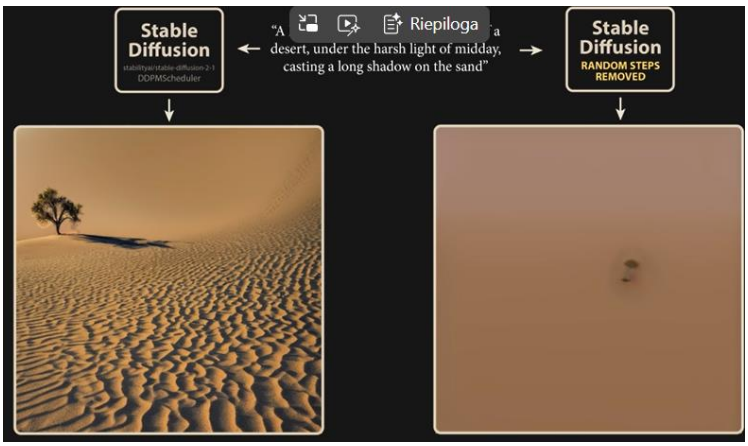
1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

These are the training and image generation algorithms from the Berkeley team's paper.

The first thing that the team **added random noise** to images *not just during training*, but also *during image generation*. Algorithm 2 tells us that when generating new images, at each step, after our neural network predicts a less noisy image (scaled model output), we need to add random noise to this image (random noise image) before passing it back into our model (output image).



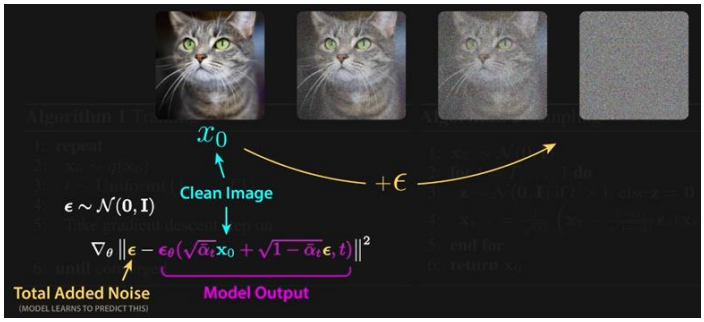
This added noise turns out to matter a lot in practice. If we take a popular diffusion model like stable diffusion 2 and use the Berkeley team's image generation approach, known as DDPM sampling, we can get some really nice images.
Here's the image we get when prompting the model with this prompt, asking for a tree in the desert. Now, if we remove the line of code that adds noise at each step of the generation process, we end up with a tiny sad blurry tree.



How is it that adding random noise while generating images leads to better quality, sharper images?

The second thing is that the team wasn't training models to reverse a single step in the noise addition process. Instead, the team takes an initial clean image, which they call X_0, and adds scaled random noise to the image, which they call epsilon. And from here, they train the model to predict the total noise that was added to the original image.

So, the team is effectively asking the model to *skip all the intermediate* steps and make a *prediction* about the original image.

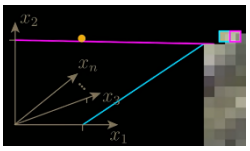


Intuitively, this learning task seems much more difficult to me than just learning to make a noisy image slightly less noisy. The Berkeley team's paper and approach was a landmark result that put diffusion on the map.

Why does adding random noise while generating images and training the model like this work so well?
The DDPM paper draws on some fairly complex theory to arrive at these algorithms. It turns out that there's a different but mathematically equivalent way of understanding what diffusion models are really learning that we can use to get a visual and intuitive sense for why the DDPM algorithms work so well.

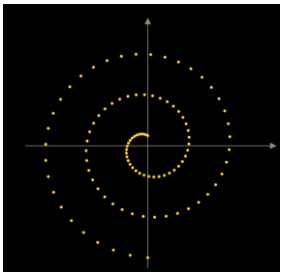
The key will be thinking of diffusion models as learning a **time-varying vector field**. This perspective also leads to a more general approach called **flow-based models**, which have become very popular recently.

To see how diffusion models learn this time-varying vector field, let's temporarily simplify our learning problem. One way to think about an image is as a point in high-dimensional space, where the intensity value of each pixel controls the position of the point in each dimension.

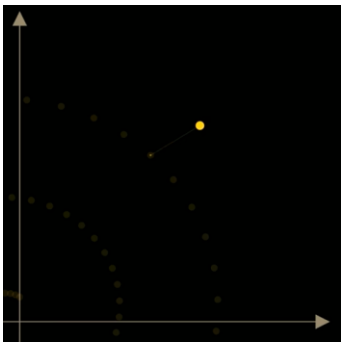


If we reduce the size of our images to only two pixels, we can visualize the distribution of our images by plotting the pixel intensity value of our first pixel on the x-axis of scatterplot and the pixel intensity of our second pixel on the y-axis. So, an image with a black first pixel and a white second pixel would show up at x equals zero and y equals one on our scatterplot. And an all-white image would be at one, one, and so on.

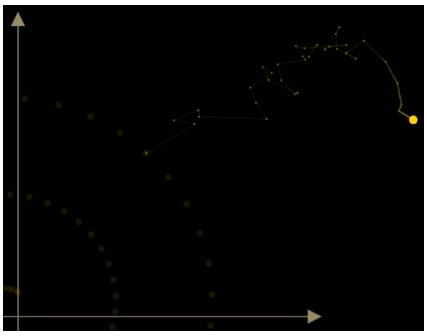
Now, real images have a very specific structure in this high-dimensional space. Let's create some structure for our points in our lower two-dimensional space for our diffusion model to learn. The exact structure we choose doesn't matter too much at this point. Let's start with a spiral shape like this.

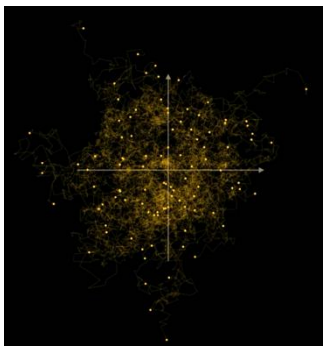


The core idea of diffusion models, adding more and more noise to an image and then training a neural network to reverse this process, looks really interesting from the perspective of our 2D toy data. When we add random noise to an image, we're effectively changing each pixel's value by a random amount. In our toy 2D dataset, where the coordinates of a point correspond to that image's pixel intensity values, adding random noise is equivalent to **taking a step in a randomly chosen direction**.



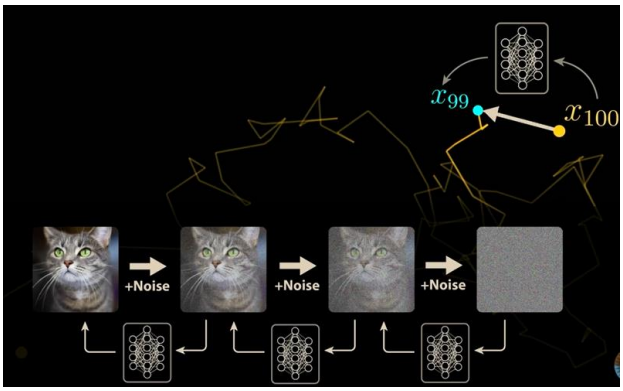
As we add more and more noise to our image, our point goes on a random walk. This process is equivalent to the Brownian motion that drives diffusion processes in physics and is where diffusion models get their name.



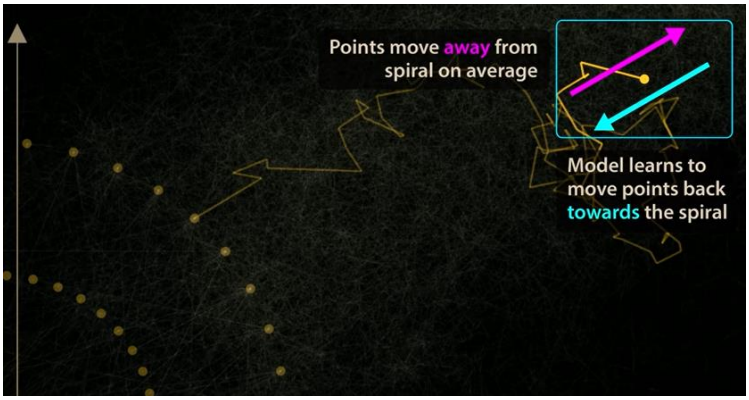


From here, it's pretty wild to think about what we're asking our diffusion model to do. Our model will see many different random walks from various starting points in our dataset, and we're effectively asking our model to reverse the clock, removing noise from our images by letting it play these diffusion processes backwards, starting our points from random locations and recovering the original structure of our dataset. How can our model learn to reverse these random walks?

If we consider the specific point at the end of this 100-step random walk, in our naive diffusion modeling approach, where we ask our model to denoise images a single step at a time, this is equivalent to giving our model the coordinates of the final 100th point in our walk, and asking our model to predict the coordinates of our point at the 99th step.

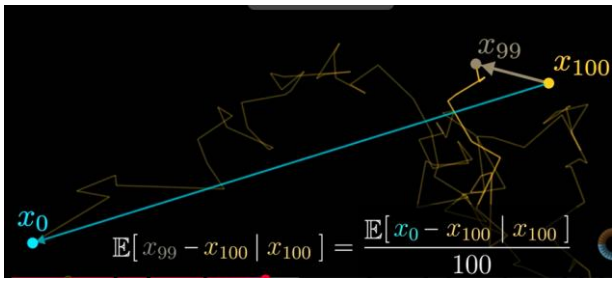


Although the direction of our 100th step is chosen randomly, there will be some signal in aggregate for our model to learn from here. Given enough training points, we expect many diffusion paths to go through this neighborhood (blue box), and on average our points will be diffusing away from our starting spiral, so our model can learn to point back towards our spiral.



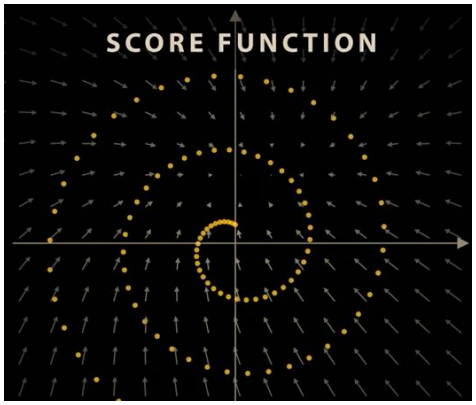
We can now see why the Berkeley team's training objective works so well. Instead of training the model to remove noise from images one step at a time, this would correspond to predicting the coordinates of the 99th step given the 100th, the team instead trained the model **to predict the total noise** added across the entire walk. On our plot, this is the vector pointing from our 100th step back to the original starting point of the walk.

It turns out that we can prove that learning to predict the noise added in the final step of our walk ($E[x_{99}-x_{100} \mid x_{100}]$) is mathematically equivalent to learning to predict the total noise added, divided by the number of steps taken.



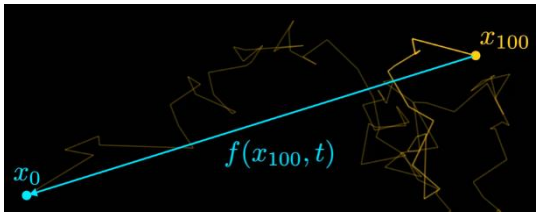
This means that when our model learns to reverse a single step, although our training data is noisy, we expect our model to ultimately **learn to point back towards x_0**. By instead training our model to directly predict the vector pointing back towards x_0, we're significantly reducing the variance of our training examples, allowing our model to learn much more efficiently, without actually changing our underlying learning objective.

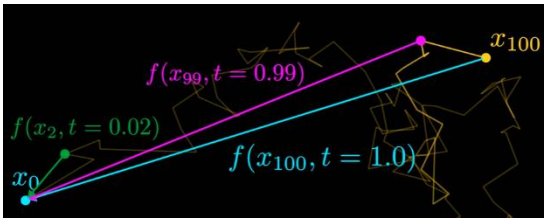
So for each point in our space, our model learns the direction pointing back towards the original data distribution.



This is also known as a **score function**, and the intuition here is that the score function points us towards more likely, less noisy data.

Now, in practice, these learned directions depend heavily on how much noise we add to our original data. After 100 steps, most of our points are far from their starting points, so our model learns to move these points back in the general direction of our spiral. However, if we train our model on examples after only one diffusion step, we end up with a much more nuanced vector field, pointing to the fine structure of our spiral. There turns out to be a clever solution to this problem. Instead of just passing in the coordinates of our point into our model, which we'll write here as a function f, we can also pass in a time variable that corresponds to the number of steps taken in our random walk.



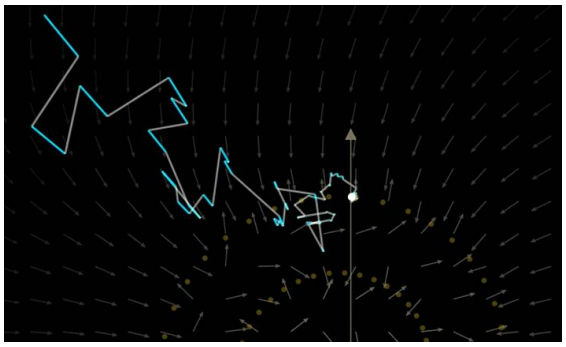


If we set t equal to 1 at our 100th step, then t would equal 0.99 at our 99th step, and so on. Conditioning our models on time like this turns out to be essential in practice, allowing our model to learn coarse vector fields for large values of t, and very refined structures as t approaches 0.

After training, we can watch the time evolution of our model. We see this really interesting behavior as t approaches 0.4. Our learned vector field suddenly transitions, from pointing towards the center of the spiral to pointing towards the spiral itself. It feels like a phase change.

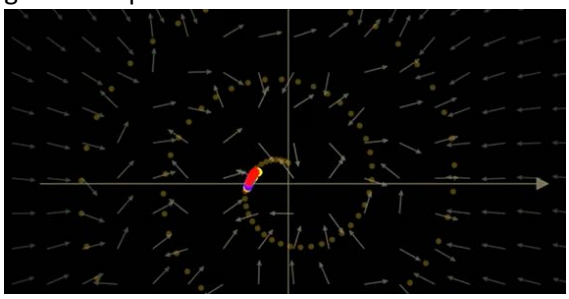
We're now in a great position to resolve the final mystery of the DDPM paper. How is it that adding random noise at each step while generating images leads to better quality, sharper images? Let's follow the path of a single point guided by the DDPM image generation algorithm. On our 2D dataset, generating an image is equivalent to starting at a random location and working our way back to our spiral. Starting at a randomly chosen location of x equals minus 1.6 and y equals 1.8, our model's vector field points us back towards our spiral. Following the DDPM algorithm, we take a small step in the direction returned by our model, and add scaled random noise, which effectively moves our point in a random direction. We'll color the steps driven by our diffusion model in blue, and our random steps in grey. Note that the scale of the random step may seem large, but following our DDPM algorithm, the size of our random steps will come down as we progress.

Repeating this process for 64 steps, our particle jumps around quite a bit due to both our learned vector field changing and our random noise steps, but ultimately lands nicely on our spiral.



Repeating this process for a point cloud of 256 points, our reverse diffusion process starts out looking like absolute chaos, but does converge nicely, with most points landing on our spiral.

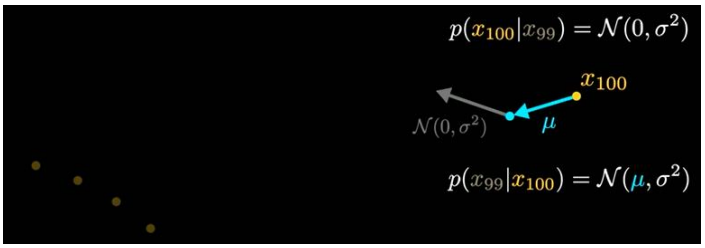
Now, what happens if we **remove** the noise addition steps? Running our reverse diffusion process again without the random noise step, all of our points quickly move to the centre of our spiral, and then make their way towards a single inside edge of the spiral.



This result can help us make sense of why we saw a sad blurry tree earlier when we removed this random noise step. Instead of capturing our full spiral distribution, as we did when we included a noise step, all of our generated points end up close to the center or average of our spiral (coloured dots) . In the space of images, averages look blurry. Conceptually, we can imagine different parts of our spiral corresponding to different images of trees in the desert. And when we remove the random noise steps from our generation process, our generated images end up in the center or average of these images, which looks like a blurry mess.

Now, note that the analogy between our toy dataset and high dimensional image dataset breaks down a bit here. If all the points on our spiral correspond to realistic images, since our generated points do still end up landing on our 2D spiral, we would expect these generated points to still look like real images, but likely with less diversity than we would want. However, in the high dimensional space of images, it appears that our image generation process doesn't quite make it to the manifold of realistic images, resulting in a blurry non-realistic image. This prediction of the average is not a coincidence.

It turns out that we can show mathematically that our model learns to point to the mean or average of our dataset, conditioned on our input point and the time in our diffusion process. One way to arrive at this result is to show that given the noise we add in our forward process is Gaussian, for sufficiently small step sizes our reverse process will also follow a Gaussian distribution, where our model actually learns the mean of this distribution. Since our model just predicts the mean of our normal distribution, to actually sample from this distribution, we need to add zero mean Gaussian noise to our model's predicted value, which is precisely what the DDPM image generation process does when we add random noise after each step.

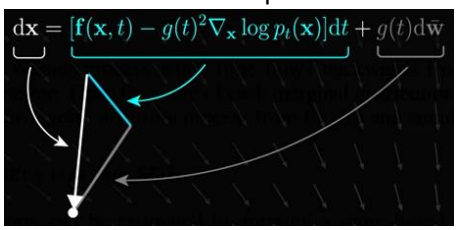


We can see this mean learning behavior most clearly early in our reverse diffusion process, when t is close to 1 and our training points are far from our spiral. Our model's learned vector field points towards the center or average of our dataset. So adding random noise during image generation falls nicely out of theory, and in practice prevents all our points from landing near the center or average of our dataset.

The DDPM paper put diffusion models on the map as a viable method of generating images, but the diffusion approach did not immediately see widespread adoption. A key issue with the DDPM approach at the time was the high compute demands of the large number of steps required to generate high quality images, since each step required a complete pass through a potentially very large neural network.

A few months later, a pair of papers from teams at Stanford and Google showed that it's remarkably possible to generate high quality images without actually adding random noise during the generation process, significantly reducing the number of steps required.

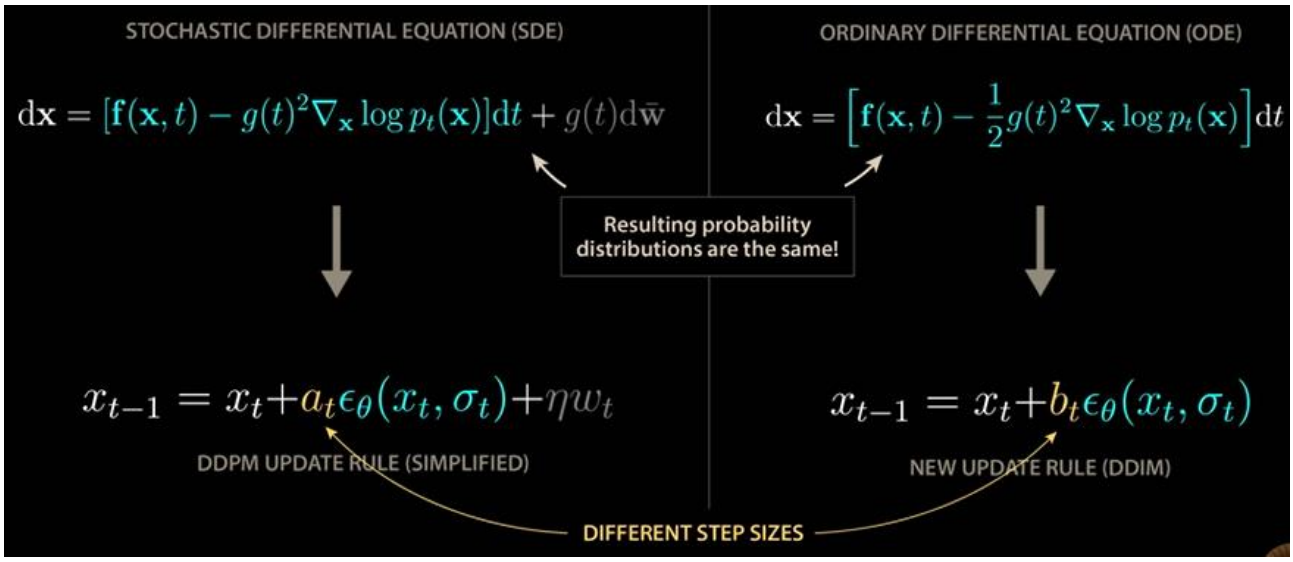
The DDPM image generation process we've been looking at can be expressed using a special type of differential equation known as a stochastic differential equation.



This first term represents the motion of our point driven by our model's vector field, and the second term represents the random motions of our point. Adding these terms together, we get the overall motion of our point at each step, dx.

From here, we can consider how the distribution of all of our points evolves over time, where the motion of each point is governed by this stochastic differential equation. This problem has been well studied in physics. Using a key result from statistical mechanics known as the Fokker-Planck equation, the Google Brain team showed that there's another differential equation, this time an ordinary differential equation with no random component, that results in the same exact final distribution of points as our stochastic differential equation.

This result gives us a new algorithm for generating images using our model's learned vector fields that does not require taking random steps along the way. Exactly how our ordinary differential equation maps to an image generation algorithm is a bit technical. The key result here though, is that we end up with something that looks very similar to our DDPM image generation process, but without the random noise addition at each step, and with a new scaling for the sizes of steps that we take. This approach is generally known as DDIM.



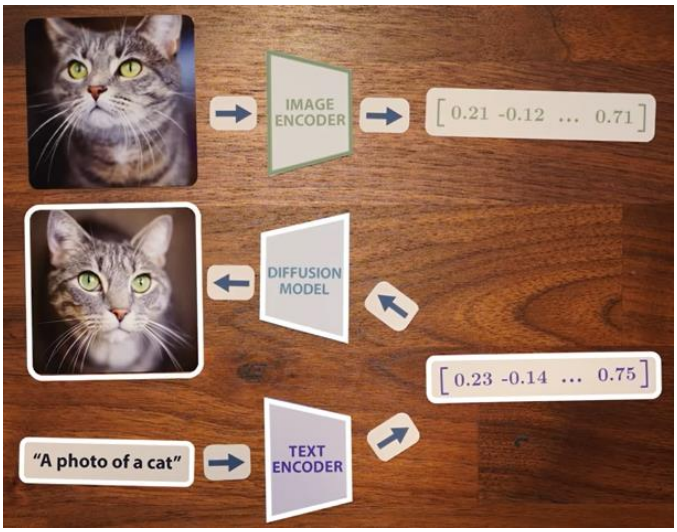
The scaling of our step sizes, and especially how these step sizes vary throughout a reverse diffusion process, matters a lot in practice.

When we just removed the random noise steps from our DDPM generation algorithm earlier, all of our points ended up near the mean of our data, and we saw blurry results for our generated images. Switching to our DDIM approach, we now have smaller scaling for our step sizes that allow our trajectories to better follow the contour lines of our vector field, and land nicely on the correct spiral distribution. And applying our DDIM algorithm to our tree in the desert example, we're now able to generate nice results. Comparing to our original DDPM algorithm that required random steps, DDIM remarkably does not require any changes to model training, but is able to generate high quality images in significantly fewer steps, completely deterministically.

Note that the theory does not tell us that our individual images or points on our spiral will be the same, but instead that our **final distribution of points or images will be the same**, regardless of whether we use our stochastic DDPM algorithm or our deterministic DDIM algorithm.

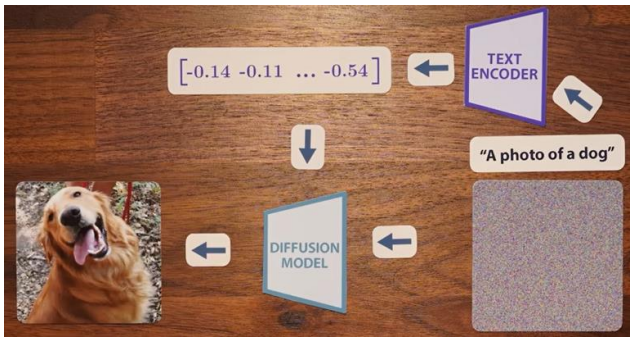
The WAN model we saw earlier uses a generalization of DDIM called **flow matching**. By early 2021, it was clear that diffusion models were capable of generating high quality images, and thanks to image generation methods like DDIM, it was possible to generate these images without using enormous amounts of compute. However, our ability to steer the diffusion process using text prompts was still very limited. Earlier, we saw how CLIP was able to learn a powerful shared representation of images and text by concurrently training image and text encoder models. However, these models only go one way, converting text or images into embedding vectors. These two problems potentially fit together in a really interesting way.

Diffusion models are able to potentially reverse the CLIP image encoder, generating high quality images, and the output vector of the CLIP text encoder could be used to guide our diffusion models toward the images or videos that we want. So the high level idea here is that we could pass in a prompt into the CLIP text encoder to generate an embedding vector, and use this embedding vector to steer the diffusion process towards the image or video of what our prompt describes.



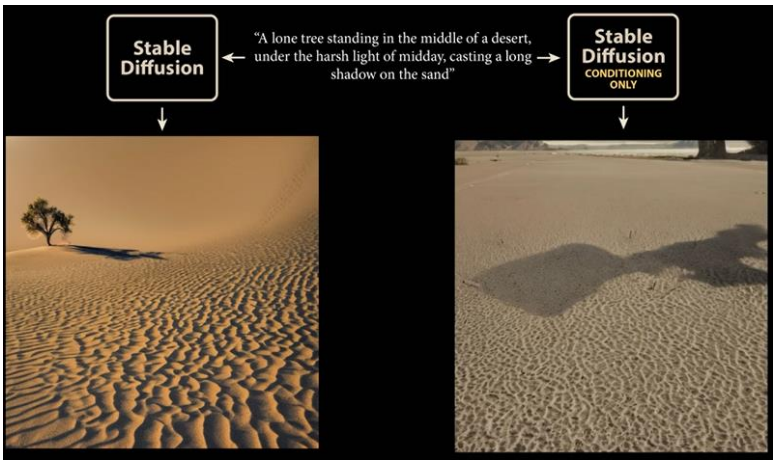
A team at OpenAI did exactly this in 2022. Using image and caption pairs to train a diffusion model to invert the CLIP image encoder. Their approach yielded an incredible level of prompt adherence, capturing an unprecedented level of detail from the input text. The team called their method unCLIP, but their model is better known by its commercial name, DALI2. But how do we actually use the embedding vectors for models like CLIP to steer the diffusion process?

One option is to simply pass our text vector as another input into our diffusion model, and train as we normally would to remove noise. If we train our diffusion model using image and caption pairs, as the OpenAI team did, the idea here is that the model will learn to use the text information to more accurately remove noise from images, since it now has more context about the image that it's learning to denoise. This technique is called **conditioning**.



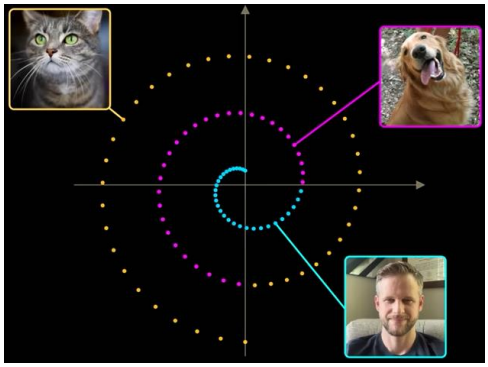
We used a similar approach earlier, when we conditioned our toy diffusion model on the number of time steps elapsed in the diffusion process, allowing the model to learn coarse structure for large values of t, and finer structures as our training samples get closer to our original spiral. Interestingly, there turns out to be a variety of ways we can pass in the text vector into our diffusion model. Some approaches use a mechanism called cross-attention to couple image and text information. Other approaches simply add or append the embedded text vector to our diffusion model's input, and some approaches pass in text information in multiple ways at once. Now it turns out that conditioning alone is not enough to achieve the level of prompt adherence that we see in models like DALI2.

If we take the stable diffusion tree in the desert example we've been experimenting with, and only condition our model with our text inputs, the model no longer gives us everything we ask for. We get a shadow in a desert, but no tree.



Note that stable diffusion was developed by a team at Heidelberg University around the same time as DALI2, and works in a similar way, but is open source.

It turns out that there's one more powerful idea that we need to effectively steer our diffusion models. We can see this idea in action by returning to our toy dataset one last time. If our overall spiral corresponds to realistic images, then different sections of our spiral may correspond to different types of images.

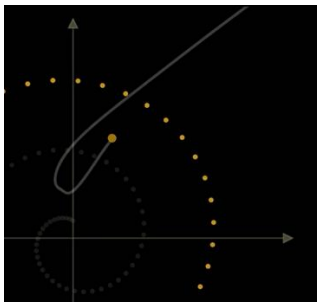


Let's say this inner part is images of people, this middle part is images of dogs, and this outer part is different images of cats.

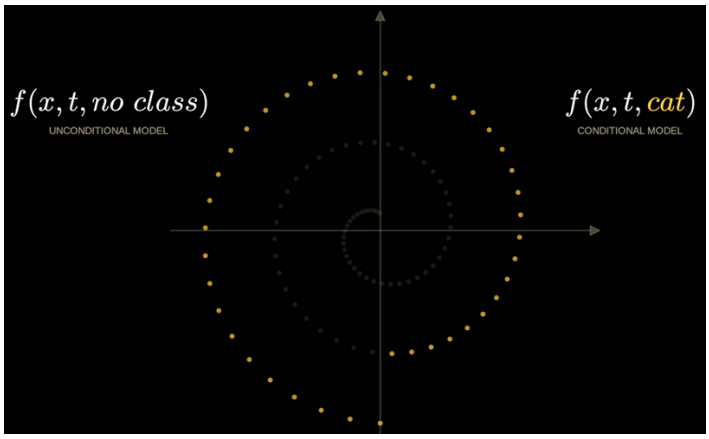
Now let's train the same diffusion model we trained earlier, but in addition to passing in our starting coordinates and the time of our diffusion process, we'll also pass in the points class. Person, cat, or dog.

This extra signal should allow our model to steer points to the right sections of our spiral, based on each points class. Running our generation process, after assigning person, dog, or cat labels to each point, we see that we're able to recover the overall structure of our dataset, but the fit is not great, and we see some confusion here between people and dog images. Part of the problem here is that we're asking our model to simultaneously learn to point to our overall spiral of realistic images, and toward specific classes on our spiral.

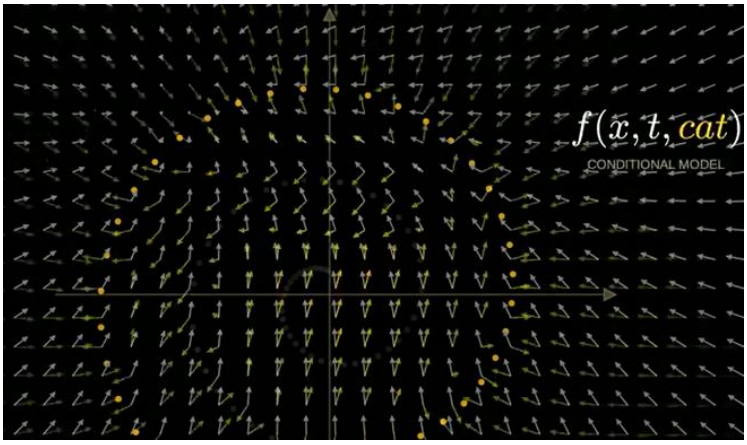
If we consider this cat point for example, it starts off heading towards the center of our spiral, and as our class conditioned vector field shifts to point towards a cat region of our spiral, our point moves towards this part of the spiral, but it doesn't quite make it.



The modeling task of generally matching our overall spiral has overpowered our model's ability to move our point in the direction of a specific class. Now, is there a way to decouple and maybe even control these two factors? Remarkably, it turns out that we can. The trick is to leverage the differences between the unconditional model that is not trained on a specific class, and a model that is conditioned on specific classes. We could do this by training two separate models, but in practice it's more efficient to just leave out the class information for a subset of our training examples. We now have the option of effectively passing in no class or text information into our model, and getting back a vector field that points towards our data in general, not towards any specific class.



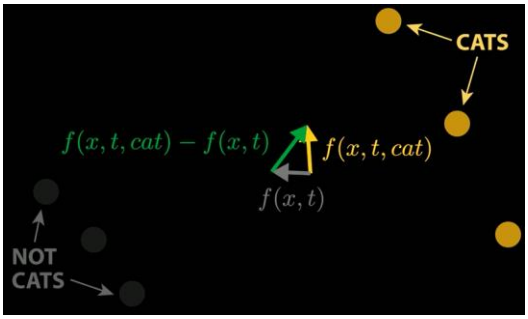
We can visualize these two vector fields together. Here the gray vectors show our diffusion model points when we don't pass in any class information, and these yellow vectors show when our model is conditioned on the cat class.



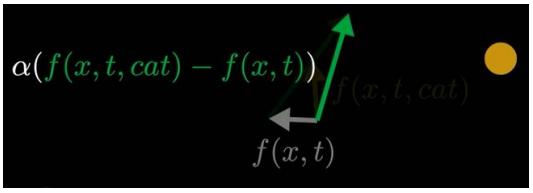
For large values of our diffusion time variable when our training data is far from our spiral, our two vector fields basically point in the same direction, roughly towards the average of our spiral. But as time approaches

zero, our vector fields diverge, with our cat conditioned vector field pointing more towards the outer cat portion of our spiral. Now that we have these two separate directions, we can use their differences to push our points more in the direction of the class we want.

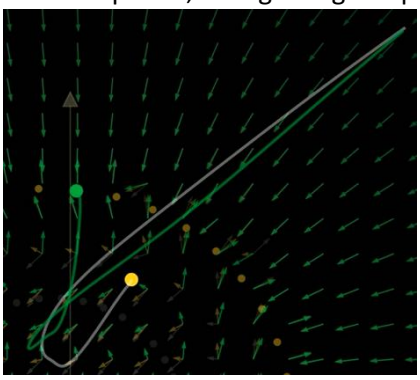
Specifically, we take our yellow class conditioned vector and subtract our gray unconditioned vector. This gives us a new vector pointing from the tip of our unconditioned vector to the tip of our conditioned vector. The idea from here is that this direction should point more in the direction of our cat examples, now that we've removed the direction generally pointing towards our data.



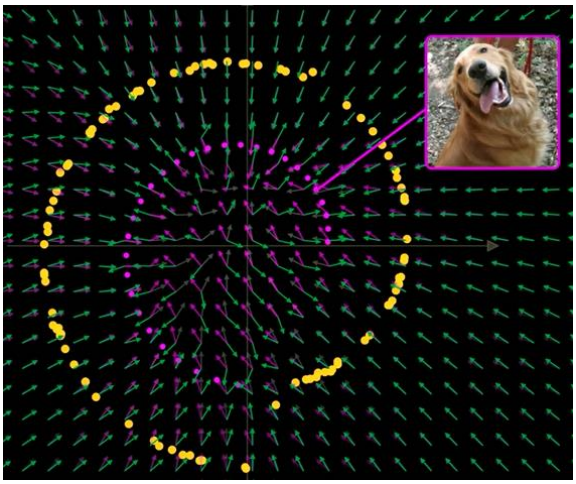
We can now amplify this direction by multiplying by a scaling factor, alpha, and replace our original conditioned yellow vector with a vector pointing in this new direction.



Let's follow the trajectory of the same cat point we saw earlier that didn't quite make it onto our spiral. We'll roll back our diffusion time variable and start a new green point from the same starting location. If we use our new green vectors to guide the diffusion process instead of our original yellow vectors, the difference between our gray arrows that point towards the center of our spiral and yellow vectors that start pointing us back towards our cat part of the spiral are amplified, now guiding our point to land nicely on our spiral.



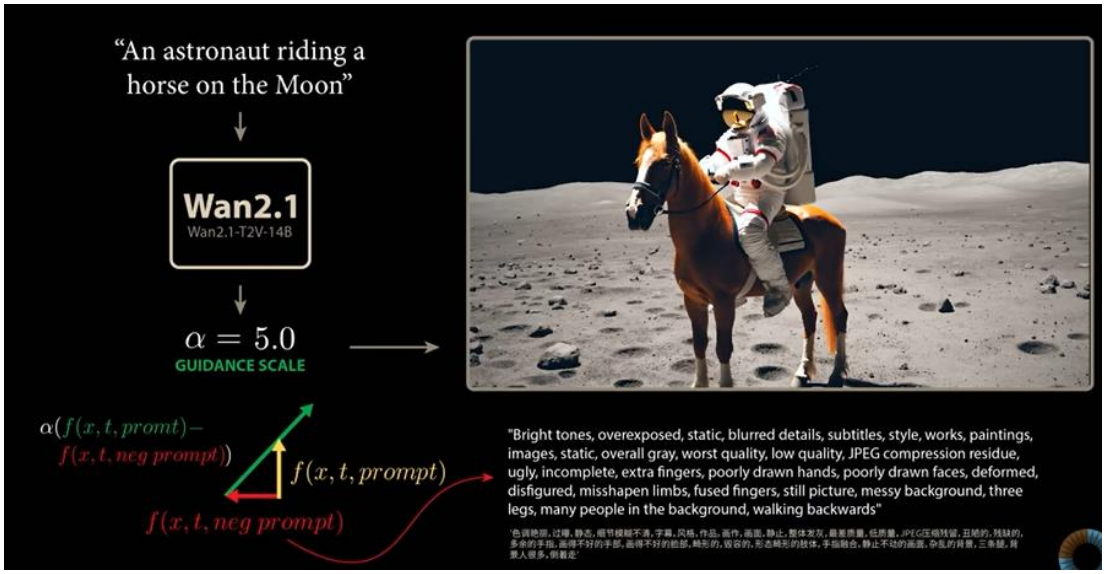
This approach is called classifier-free guidance. Using our new green vectors to guide a set of cat points, we see a nice tight fit to our spiral for this class. Switching to our dog class, our unconditional gray vector field stays the same, but our dog conditioned model outputs, shown in magenta, now point us more towards the dog part of our spiral. And adding guidance amplifies this learned direction.



Using our guided vectors and running our generation process, we see a nice fit for our dog points. Finally, we get a third vector field for our people examples that again results in nice convergence to our spiral.

Classifier-free guidance works remarkably well and has become an essential part of many modern image and video generation models. Earlier, we saw that if we only conditioned our stable diffusion model, our image would have a desert and a shadow, but no tree that we asked for in the prompt. If we add classifier-free guidance to this model, once we reach a guidance scale alpha of around 2, we start to actually see a tiny tree in our images. And the size and detail of our tree improve as we increase our scaling factor, alpha. The fact that this works so well is remarkable to me. As we use guidance to point our stable diffusion model's vector field more in the direction of our prompt, our tree literally grows in size and detail in our images.

Our WAN video generation model takes this guidance approach one step further. Instead of subtracting the output of an unconditioned model with no text input, the WAN team uses what's known as a negative prompt, where they specifically write out all the features they don't want in their video, and then subtract the resulting vector from the model's conditioned output and amplify the result, steering the diffusion process away from these unwanted features. Their standard negative prompt is fascinating, including features like extra fingers and walking backwards, and interestingly is actually passed into their text encoder in Chinese.



Here's a video generated using the same astronaut on a horse prompt we used earlier, but without the negative prompt. It's really interesting to see how the parts of the scene get cartoonish and no longer fit together.

Since the publication of the DDPM paper in the summer of 2020, the field has progressed at a blistering pace, leading to the incredible text-to-video models that we see today. Of all the interesting details that make these models tick, the most astounding thing to me is that the pieces fit together at all. The fact that we can take a trained text encoder from clip or elsewhere and use its output to actually steer the diffusion process, which itself is highly complex, seems almost too good to be true. And on top of that, many of these core ideas can be built from relatively simple geometric intuitions that somehow hold in the incredibly high dimensional spaces these models operate in. The resulting models feel like a fundamentally new class of machine. To create incredibly lifelike and beautiful images and video, you no longer need a camera, you don't need to know how to draw or how to paint, or how to use animation software. All you need is language.